

E0 Variable-Agent Compatibility Gate

Methods, Results, and Promotion Decision

AlemDICE Research Engineering Report

22 July 2026

Abstract

This report records E0, a provider-free compatibility gate for a staged study of agent-population scaling and decentralized team recruitment in Alem. The canonical evaluator was exercised with one, two, three, four, and six physical agents, two deterministic seeds, and ten ticks per episode. Scripted agents selected `Noop` and emitted deterministic messages, isolating evaluator and environment correctness from model behavior. All ten episodes completed. Client cardinality, observation and trajectory dimensions, specialization cycling, targeted handoffs, action parsing, communication accounting, artifact serialization, deterministic replay, and representative full-world rendering passed at every population. There were zero model calls, provider requests, and transport errors. E0 therefore removes the engineering blocker for a paid Source scaling screen, including a gated six-agent extension. It is not evidence that larger teams perform better or coordinate effectively.

1 Question and claim boundary

The immediate question was whether the existing Alem language-agent evaluation stack can safely vary the number of physical agents before inference budget is spent on behavioral experiments. This matters because changing population affects observation length, action indexing, specialization assignment, communication fan-out, trajectory axes, and renderer assumptions. A failure in any of these mechanisms could be misread later as a language-model or coordination failure.

E0 is deliberately an *engineering gate*. It tests the execution and audit path, not policy quality. The accepted claim is:

At source commit `587e717`, the canonical evaluator completed short, provider-free episodes at $N \in \{1, 2, 3, 4, 6\}$ and satisfied all preregistered compatibility assertions.

E0 does not test task performance, useful communication, model reasoning, provider parallelism, dynamic recruitment, context management, or long-horizon role alignment.

2 Methods

2.1 Design and configuration

The test matrix crossed five population sizes, $N \in \{1, 2, 3, 4, 6\}$, with seeds 13000 and 13001. Every episode used `alem/default`, Easy coordination, soft specialization, baseline communication topology, one episode worker, and a horizon of ten ticks. Scratchpad, chain-of-thought, image history, debrief generation, and W&B were disabled. The resolved configuration for each population was saved beside its artifacts.

Configuration began from the validated `fake_smoke` profile. The harness created exactly one client configuration slot for each physical agent and gave each slot a unique cache key. No client was called: a

test-only agent factory supplied deterministic actors directly to the same `EvaluatorManager` used by language-model evaluations. This preserved the normal reset, step, action validation, communication routing, accounting, trajectory, state, debug, and terminal-summary paths while guaranteeing zero provider traffic.

2.2 Scripted policy

At every tick, every actor returned the canonical action `Noop`. Actor i at tick t also emitted the 30-byte message

`EO|AGENT= $i$ |TICK= $t$ |STATE=ACTIVE.`

Messages were delivered by the existing one-tick baseline broadcast route. Because the policy is deterministic and intentionally task-agnostic, any failure is attributable to the variable-agent infrastructure rather than sampling or semantic model behavior.

2.3 Assertions

The harness applied the following checks after each population:

1. **Execution and cardinality.** Two complete episodes existed; each contained exactly ten steps and reported the requested N . Every observation, action, reward, text-observation, and text-action trajectory had a ten-step time axis and an N -agent axis.
2. **Action interface.** `Noop` parsed on every attempted action. For every ordered pair of distinct agents (i, j) , the label `Give to Agent j` round-tripped through the agent-specific action mapper to the correct handoff slot. This yields $N(N - 1)$ checked mappings.
3. **Roles.** Saved state used the expected repeating environment specialization sequence Warrior, Forager, Miner, with encoded values $(2, 1, 3)$.
4. **Communication.** With $T = 10$ ticks per seed, emitted messages had to equal NT , and delivered copies had to equal $N(N - 1)T$. Since each payload was 30 bytes, delivered byte counts were also exact.
5. **Provider isolation.** Model calls, provider requests, and transport errors all had to remain zero.
6. **Artifacts and replay.** Episode JSON, CSV, debug JSONL/HTML, step text, trajectory NPZ, compressed state bundle, attempt ledger, and resolved configuration had to exist. Every state bundle contained ten snapshots. A stable projection of the first saved state was SHA-256 hashed and compared with an independently reset environment using the same seed.

The stable state projection included the world map, item map, player positions, player specializations, and coordination map. Equality of these hashes is a targeted determinism assertion rather than a claim that arbitrary Python pickle bytes are platform-independent.

2.4 Renderer audit

After the canonical run, the first saved state from seed 13000 at each population was passed through `render_full_world_replay.py`. Each smoke artifact encoded one frame with `libx264`. `ffprobe` independently verified an H.264 stream, one frame, and dimensions 768×832 . Renderer hashes are recorded in the machine-readable published summary.

2.5 Execution and provenance

The run used Python 3.12.13 on Linux-6.17.0-40-generic-x86_64-with-glibc2.42. It began at 2026-07-23 06:24:16 UTC and ended at 06:28:23 UTC, taking 246.64 seconds. The source branch was `research/agent-scaling-recruitment-pipeline`.

The harness source commit was 587e7176ce3b02296120e9504c1d7d609e9dcf2a. The raw manifest noted one pre-existing untracked replay outside the E0 output root. That file was neither used nor modified.

3 Results

3.1 Population-level results

Every population passed every asserted gate. Table 1 reports totals across the two seeds at each population.

N	Eps.	Ticks	Obs.	Actions	Give	Emit	Deliver	Bytes	Result
1	2	20	9,476	55	0	20	0	0	PASS
2	2	20	9,597	55	2	40	40	1,200	PASS
3	2	20	9,718	56	6	60	120	3,600	PASS
4	2	20	9,839	57	12	80	240	7,200	PASS
6	2	20	10,081	59	30	120	600	18,000	PASS

Table 1: E0 population results. “Obs.” is the flattened observation length; “Give” is the number of ordered targeted-handoff mappings checked.

Across the full matrix, E0 executed 100 environment ticks and 320 scripted agent-ticks. It emitted 320 messages and delivered 1,000 copies totaling 30,000 bytes. The evaluator saved 100 states in ten complete episodes and produced 82 canonical files before renderer smoke artifacts were added. All actions parsed, all ten saved-state replay projections matched, and all five renderer checks passed. There were zero model calls, provider requests, transport failures, or failed episodes.

3.2 Structural scaling observations

The observation length followed

$$d_{\text{obs}}(N) = 9476 + 121(N - 1),$$

showing that each additional teammate contributed 121 serialized values under this wrapper. The action count followed

$$|\mathcal{A}(N)| = 54 + \max(1, N - 1),$$

because targeted handoff slots expand with the number of peers while the single-agent interface retains its own minimum action surface.

The handoff validation count grew as $N(N - 1)$, covering both directions for every pair. More importantly for later scaling studies, baseline message delivery also grew as $N(N - 1)$ per tick: six agents generated 120 messages but 600 delivered copies over the two short episodes. This confirms the accounting needed to measure broadcast cost in E1; it does not establish that those bytes are useful.

4 Gate decision

E0 is **PASS**. No population is blocked by the tested evaluator, action, routing, serialization, deterministic replay, or rendering mechanisms. The planned E1a Source screen may therefore include $N = 1, 2, 3, 4$, and the engineering precondition for the separately labeled $N = 6$ extension is satisfied.

The promotion is narrow. Six agents remain beyond the paper’s documented one-to-four range and require explicit behavioral validation. E1 must use real model actions, paired seeds, longer horizons, per-agent normalization, task outcomes, and communication/inference cost. A positive E0 result must not be plotted as a performance data point or cited as evidence of coordination.

5 Limitations

- Noop avoids semantic and task-dependent action sequences.
- Two seeds and ten ticks are sufficient for this deterministic gate but provide no behavioral power.
- Baseline broadcast was tested; team-scoped routing and recruitment state machines do not yet exist in this checkpoint.
- Provider concurrency, response parsing under real generations, context growth, role drift, deaths, and recovery were not exercised.
- Renderer smoke used one representative state per population rather than encoding every trajectory frame.

6 Reproduction and audit artifacts

The canonical command was:

```
uv run -extra baselines-llm -python 3.12
python scripts/run_e0_agent_compatibility.py
-output outputs/alem_eval/e0_agent_compatibility_v1
```

The ignored raw source of truth is `outputs/alem_eval/e0_agent_compatibility_v1/e0_results.json`, whose SHA-256 is `7b67b56544a00df63b1e07106c9eeffe51eedc9bc62dc7d8d914af734ad8c47d`. The compact tracked summaries are `Results/e0_agent_compatibility_v1.json` and `Results/e0_agent_compatibility_v1.md`. The frozen protocol and append-only implementation log are in `reports/agent_scaling_recruitment/`.

7 Conclusion

E0 found no variable-agent infrastructure failure across the planned population ladder. The result establishes a reproducible launch point for E1 and makes the quadratic cost of unchanged baseline broadcast explicit. It does not answer the scientific question of whether more agents, restricted communication, or decentralized recruitment improves collective behavior; those questions remain assigned to the staged behavioral experiments.